Final Year Design Project System Requirements Specification

AI Based Body Measurement App

Software Design Specification Document

by

Hassan Abdullah	**BCS 22014**
H Muhammad Ertaza	**BCS 22044**
Muhammad Imran	**BCS 22045**
Dawood Rasheed	**BCS 22046**

Project Advisor:
Sir Moodser Hussain

Department of Information Technology
University of the Punjab, Gujranwala Campus, Gujranwala, Pakistan

(2025)

AI Based Body Measurement App

Executive Summary

This Software Design Specification (SDS) document explains the detailed design of the **AI-Based Body Measurement Mobile Application**, developed as a Final Year Design Project. The purpose of this application is to automatically calculate body measurements such as chest, waist, and arms using artificial intelligence, without requiring manual measuring tools.

This Software Design Specification (SDS) describes the design of the AI-Based Body Measurement App. The system allows users to capture or upload an image, detects body keypoints using an on-device pose estimation model (MoveNet/ML Kit), calculates body measurements (e.g., chest, waist, hip, limb lengths), and displays results to the user. The application supports secure authentication and storage of measurement history using Firebase, and provides dress browsing and personalized size/design recommendations. The design emphasizes on-device processing for privacy, reduced cloud reliance, and offline measurement capability, aligned with the project proposal objectives and the approved SRS.

Table of Contents

List of Tables

Tables of Figures

System Design

The AI-Based Body Measurement App is a **standalone mobile application**. Users interact with the system through their smartphone camera to capture body posture. The app processes this input using AI models and displays accurate body measurements in real time.
The system interacts with:

- Mobile device camera
- On-device AI models (MoveNet, ML Kit)
- Firebase Authentication
- Firebase Firestore database

Design Constraints

- The system must run efficiently on mid-range smartphones
- AI processing must be fast and real-time
- Images are not stored or sent to servers for privacy reasons
- Limited mobile device resources (CPU, memory)

1. Design Considerations

Assumptions and Dependencies:

- Users have a smartphone with a working camera
- Internet is required only for login and saving data
- TensorFlow Lite models are supported on the device

Limitations:

- Measurement accuracy depends on lighting conditions
- Loose clothing may affect results
- Only one person can be measured at a time

Risks:

One potential risk in the system is **incorrect user posture** during body measurement, which may lead to inaccurate results. To reduce this risk, the application provides **on-screen posture guidelines and visual instructions** that help users maintain the correct position while scanning.

Another risk is **slow performance**, especially on mid-range or older mobile devices. This issue is addressed by using **optimized and lightweight AI models**, such as MoveNet in TensorFlow Lite format, which are designed for real-time execution on mobile hardware.

A major concern in AI-based camera applications is **user privacy**. To mitigate this risk, all pose detection and measurement calculations are performed **directly on the user's device**. No images or video data are uploaded or stored on external servers, ensuring user data remains private and secure.

2. Requirements Traceability Matrix 1

Table 1: Requirements Traceability Matrix

Requirement ID	Requirement Description	Design Specification
FR-1	The system must allow user registration and secure authentication	FirebaseAuthService, User, AuthUI
FR-2	The system must capture or upload body images	CameraHandler, ImagePreprocessor
FR-3	The system must detect body keypoints	PoseDetectionService
FR-4	The system must calculate body measurements	MeasurementEngine
FR-5	The system must display measurement results	MeasurementResultUI
FR-6	The system must store measurement history	FirebaseFirestore, MeasurementRepository
FR-7	The system must provide clothing recommendations	RecommendationEngine, DressCatalogService

3. Design Models

1. Class Diagram

The UML Class Diagram represents the static structure of the AI-Based Body Measurement App. It shows the main classes involved in the system, their attributes, methods, and relationships with each other. This diagram helps understand how system entities are organized and how responsibilities are distributed among different components.

Key classes include User, CameraHandler, **PoseEstimator**, MeasurementEngine, MeasurementRecord, PoseLandmark, FirebaseService, and RecommendationEngine. The RecommendationEngine is responsible for generating personalized clothing size and design suggestions based on user measurements.

The User class manages user profile and authentication information. The CameraHandler captures images from the mobile device camera. The PoseDetectionService uses MoveNet / ML Kit to detect body keypoints. The MeasurementEngine calculates body measurements such as height, chest, waist, and hip based on detected landmarks. The MeasurementRecord stores calculated values along with timestamps, while FirebaseService handles secure storage and retrieval of user data.

The class diagram provides a clear blueprint of system components and supports an object-oriented design approach, improving modularity, maintainability, and scalability.

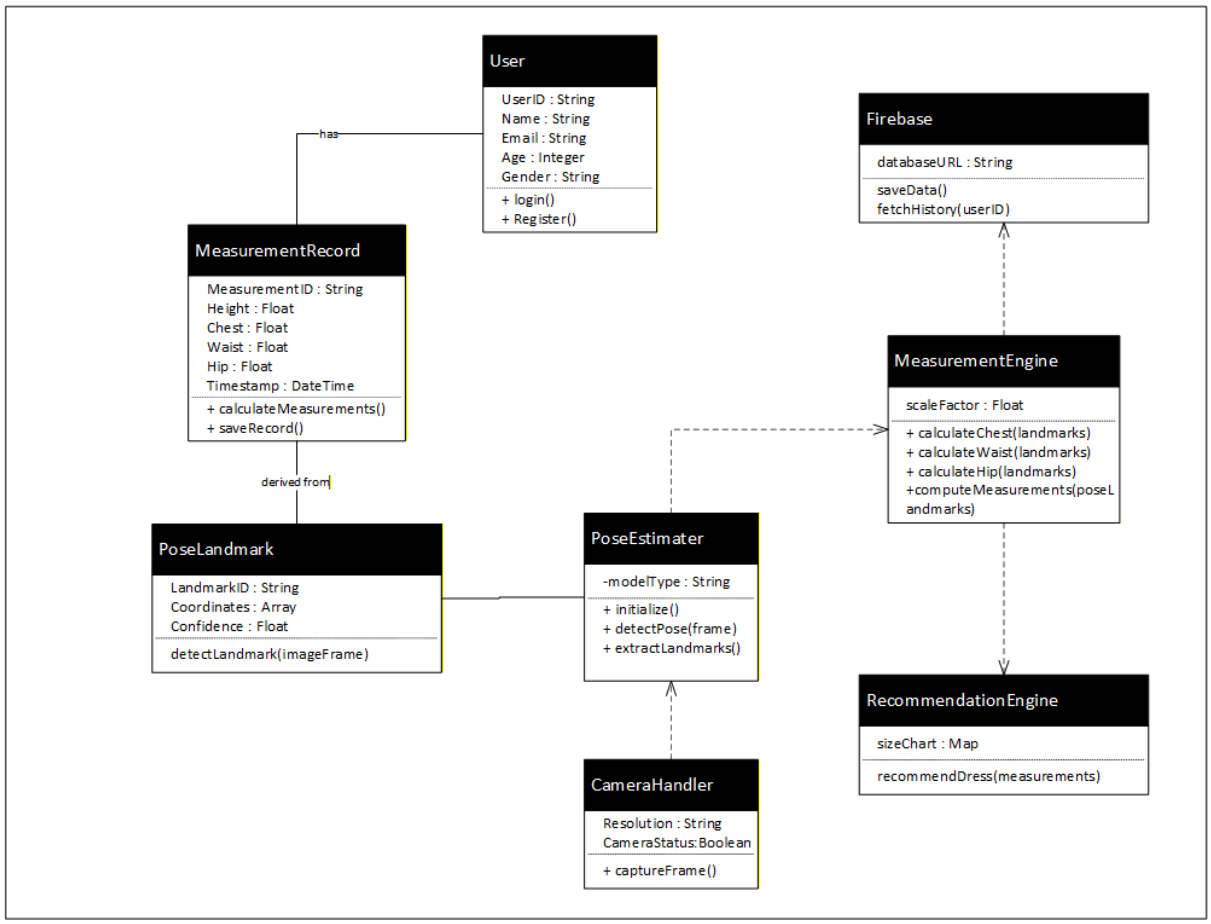

Figure 1: Class Diagram

2. Interaction Diagram (sequence)

The Interaction (Sequence) Diagram illustrates the time-ordered communication between system components during the body measurement process. It focuses on how user actions trigger system responses and how data flows between modules.

The sequence begins when the End User initiates a measurement by capturing or uploading an image. The CameraHandler captures the image and sends it to the PoseDetectionService, which uses MoveNet / ML Kit to detect body keypoints. These keypoints are passed to the MeasurementEngine, which calculates body measurements. The results are then displayed through the User Interface, and the FirebaseService stores the measurements securely.

This diagram helps visualize runtime behavior and ensures that component interactions follow the required workflow defined in the SRS.

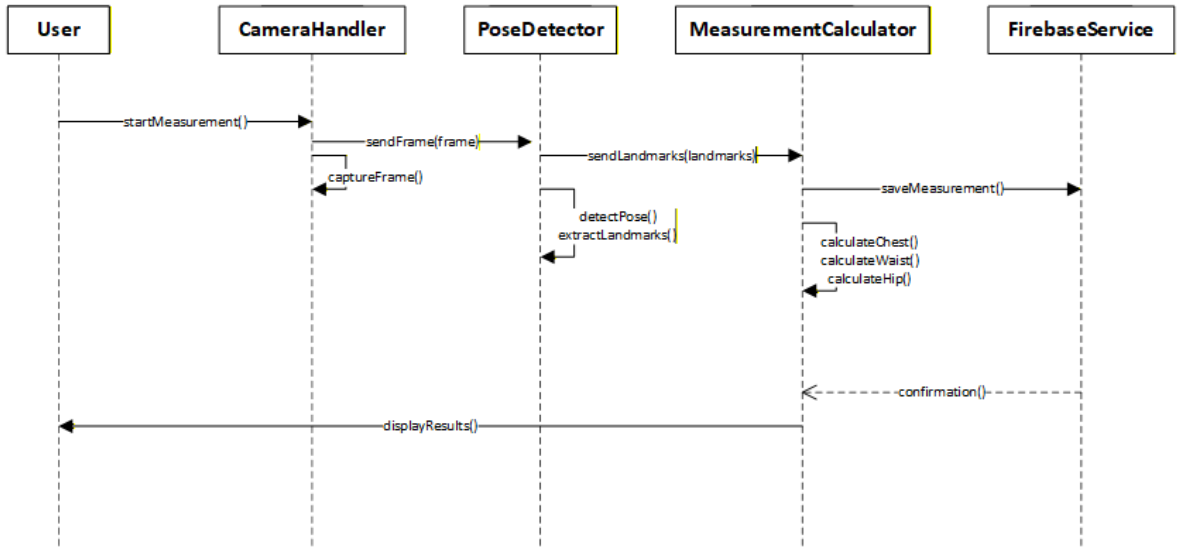

Figure 2: Sequence Diagram

3. State Transition Diagram

The State Transition Diagram describes the logical states of the AI-Based Body Measurement App and the transitions between these states during execution. It presents how the system changes state in response to user actions and internal events.

Typical states include Idle, Image Capture, Pose Detection, Measurement Calculation, Result Display, and Data Storage. Transitions occur when events such as image capture completion, successful pose detection, or measurement calculation are triggered.Additional states include *Save Measurement* and *Process Completed*, representing persistent storage and completion of the measurement workflow.

This diagram provides a design-level behavioral view of the system and helps developers understand valid system states and transitions, reducing logical errors during implementation.

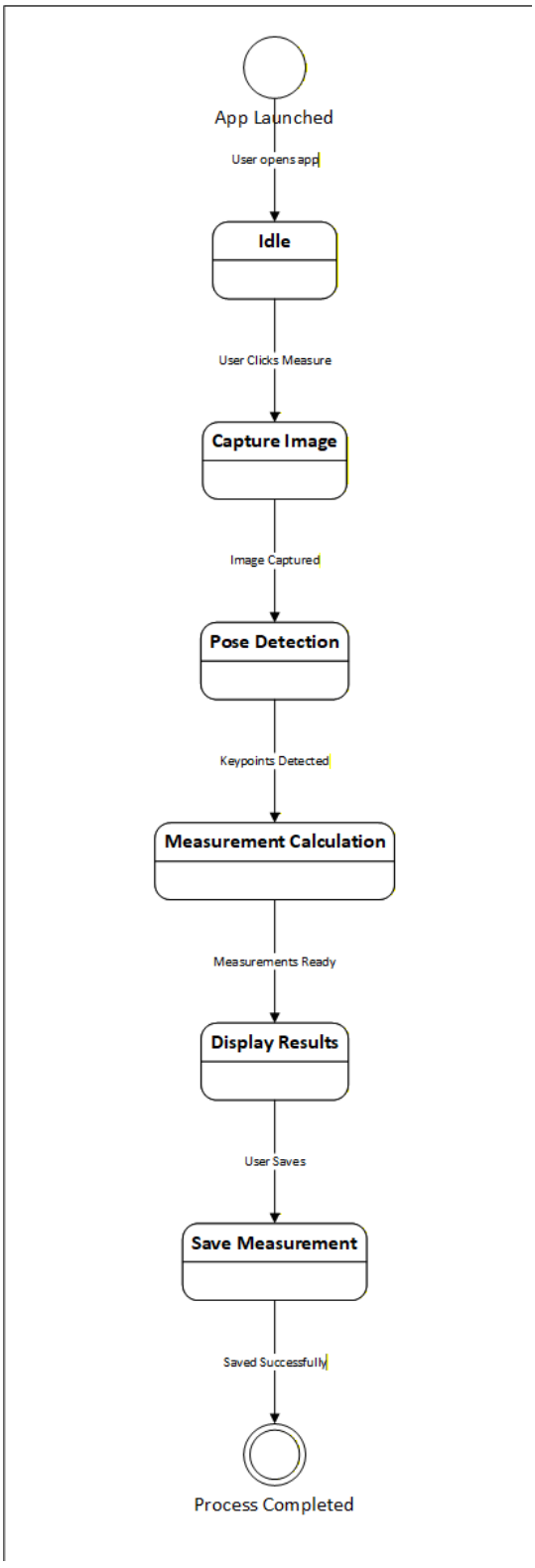

Figure 3: State Transition Diagram

3.1 Architectural Design

The AI-Based Body Measurement App follows a Layered Client–Server Architecture, ensuring clear separation of concerns, scalability, and ease of maintenance.

- The Presentation Layer manages user interactions such as login, image capture, and result display using a mobile UI framework.
- The Application Logic Layer controls workflow execution, coordinating between UI, AI processing, and data storage.
- The AI Processing Layer performs on-device pose detection using MoveNet / ML Kit and calculates body measurements.
- The Data Layer manages authentication and persistent storage using Firebase Authentication and Firebase Firestore / Realtime Database.

This architecture ensures on-device processing for privacy, efficient communication between components, and flexibility for future enhancements.

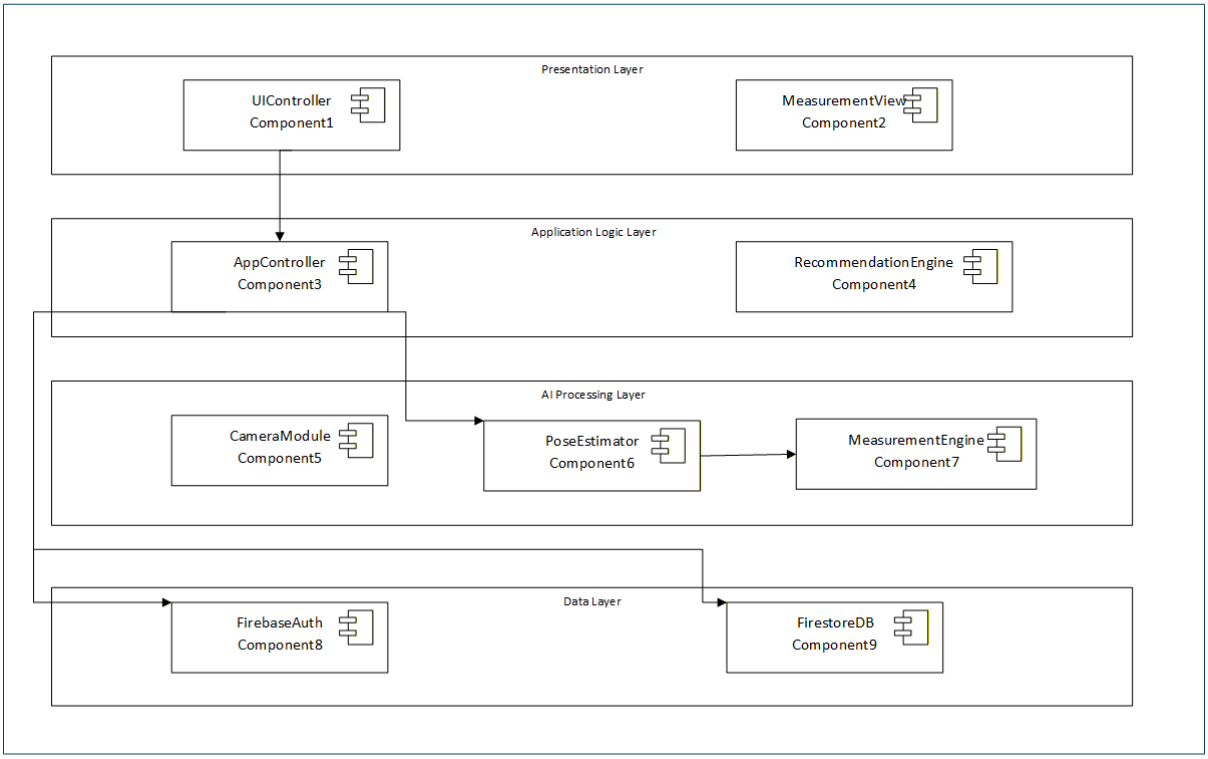

Figure 4: Layered and Client–Server Architectural Design .

3.2 Data Design

The Data Design defines how data is structured, stored, and managed within the AI-Based Body Measurement App. The primary entities include User, MeasurementRecord, and UserAuth.

User profiles and measurement data are stored in Firebase Firestore / Realtime Database using a NoSQL structure for flexibility and scalability. Each measurement record includes a unique ID, timestamp, and calculated body dimensions. Relationships between users and measurements are maintained using user identifiers.

This design ensures efficient data retrieval, secure storage, and support for historical measurement tracking.

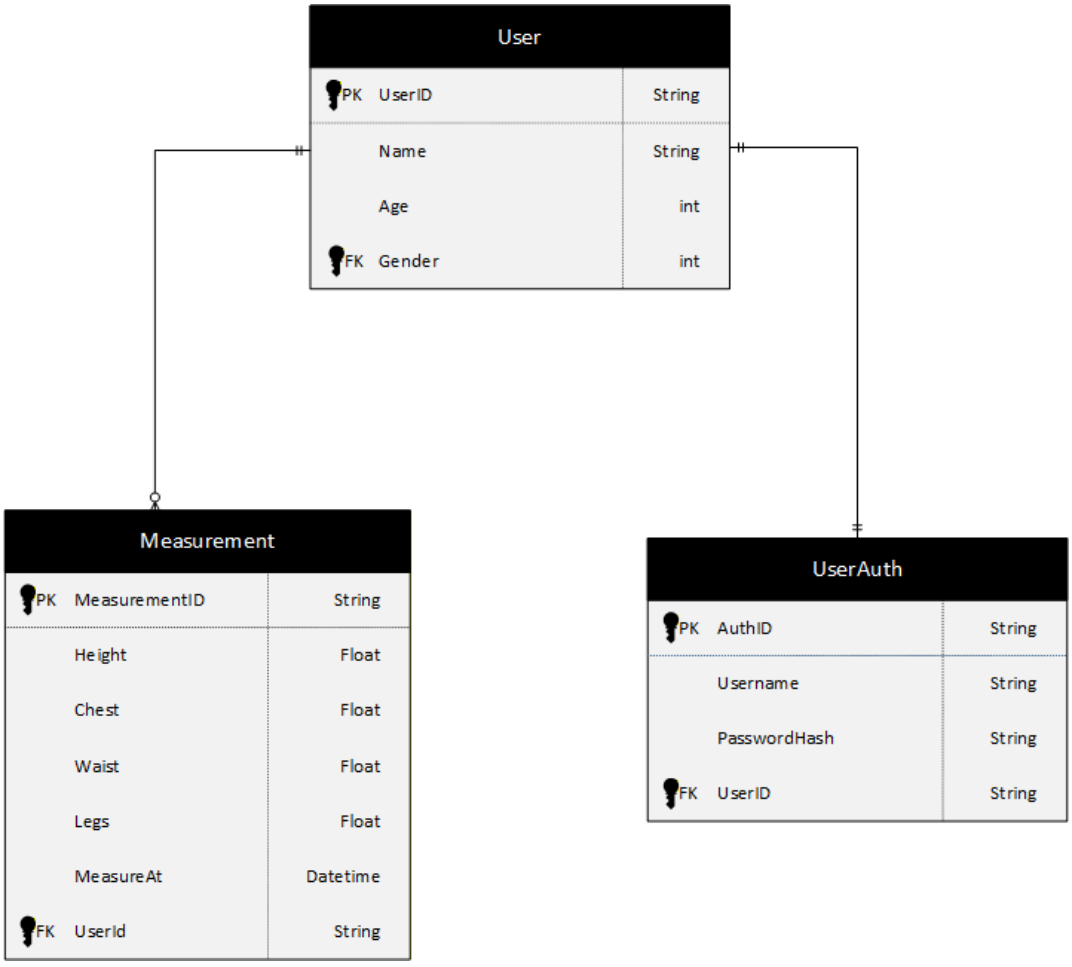

Figure 5: ERD Diagram

1.1.1. Data Dictionary

- **Alphabetical List of System Entities or Major Data with Types and Descriptions:**

Table 2: Alphabetical List of System Entities

Terminology	Description
Age	Integer. Represents the age of the user, stored as part of the user profile.
Chest	Float. Stores the chest measurement calculated by the AI measurement engine.
Confidence	Float. Indicates the confidence level of each detected pose landmark generated by MoveNet.
Coordinates	String / Array. Stores the X and Y coordinates of body landmarks detected during pose estimation.
Email	String. User's email address used for login and identification purposes.
Gender	String. Represents the gender of the user for profile completeness.
Height	Float. Stores the calculated height measurement of the user.
LandmarkID	String. Unique identifier for each pose landmark.
MeasurementID	String. Unique identifier assigned to each measurement record.
MeasurementRecord	Object. Stores calculated body measurements such as height, chest, waist, and hip.
Name	String. Stores the full name of the user.
Password	String. Encrypted password used for user authentication.
PoseLandmark	Object. Represents detected body joints used for measurement calculations.
SessionID	String. Unique identifier representing a measurement session or history entry.
Timestamp	DateTime. Records the date and time when a measurement is taken.
User	Object. Represents a registered user of the application.
UserID	String. Unique identifier assigned to each user.
Waist	Float. Stores the waist measurement calculated by the AI module.

- **Structured Approach: Functions and Function Parameters:**

Table 3: Structured Approach

Function Name	**Description**	**Parameter Name**	**Data Type**	**Parameter Role**
captureImage()	Captures an image or video frame from the device camera for	cameraFrame	ImageFrame	Stores the captured image from the device camera.

	pose detection.			
detectPose()	Detects body pose landmarks using the MoveNet model.	imageFrame	ImageFrame	Provides the input image for pose detection.
calculateMeasurements()	Calculates body measurements such as height, chest, waist, and hip using ML Kit.	poseLandmarks	List<PoseLandmark>	Contains detected body landmarks used for measurement calculation.
saveMeasurement()	Saves the calculated body measurements to the Firebase database.	measurementRecord	MeasurementRecord	Holds measurement values and related metadata for storage.
getMeasurementHistory()	Retrieves previous measurement sessions for a specific user.	userID	String	Identifies the user whose measurement history is requested.
authenticateUser()	Authenticates the user during login.	email	String	Used to identify the user account.
authenticateUser()	Authenticates the user during login.	password	String	Used to verify user credentials securely.
createUserProfile()	Creates a new user profile in the system.	userData	User	Contains user information such as name, email, age, and gender.

- **Object-Oriented (OO) Approach: Objects, Attributes, Methods, & Method Parameters:**

1. User Object :

Element	Details
Attributes	UserID (String), Name (String), Email (String), Age (Integer), Gender (String), Password (String)
Description	Represents a registered user of the application and stores profile and authentication information.
Methods	registerUser(), loginUser()
Method Parameters	registerUser(name:String, email:String, password:String, age:Integer, gender:String)loginUser(email:String, password:String)

2. MeasurementRecord Object

Element	Details
Attributes	MeasurementID (String), Height (Float), Chest (Float), Waist (Float), Hip (Float), Timestamp (DateTime)
Description	Stores body measurement values calculated by the AI module.
Methods	calculateMeasurements(), saveMeasurement()
Method Parametes	calculateMeasurements(poseLandmarks:List<PoseLandmark>)saveMeasurement()

3. PoseLandmark Object

Element	Details
Attributes	LandmarkID (String), Coordinates (String/Array), Confidence (Float)
Description	Represents detected body joints generated by the MoveNet pose detection model.
Methods	detectLandmark()
Method Parameters	detectLandmark(imageFrame:ImageFrame)

4. CameraHandler Object

Element	Details
Attributes	Resolution (String), CameraStatus (Boolean)
Description	Handles camera operations for capturing images used in pose detection.
Methods	captureImage()
Method Parameters	captureImage(): ImageFrame

3.3User Interface Design

The AI-Based Body Measurement App allows users to measure their body dimensions easily using a mobile phone camera. The system is designed to be simple so that any user can use it without technical knowledge.

When the user opens the app, they first **register or log in** using their email and password. After logging in, the user sees the main screen, where they can start a new measurement or view their previous measurement records.

To take a measurement, the user opens the **camera** and follows simple on-screen instructions to stand in the correct position. The system captures the image and uses **AI pose detection (MoveNet)** to detect body points. These points are then used by the **measurement module (ML Kit)** to calculate body measurements such as height, chest, waist, and hip.

After processing, the calculated measurements are **shown clearly on the screen**. The user also receives **feedback messages**, such as confirmation of successful measurement or guidance to adjust posture if needed. All measurement results are **automatically saved** so the user can access them later.

The user can view all past measurements in the **history section**, where results are displayed with date and time. This helps the user track body changes over time.

Overall, the system provides an **easy, accurate, and user-friendly way** to measure the body, view results instantly, and maintain a history of measurements.

3.3.1. Screen Images

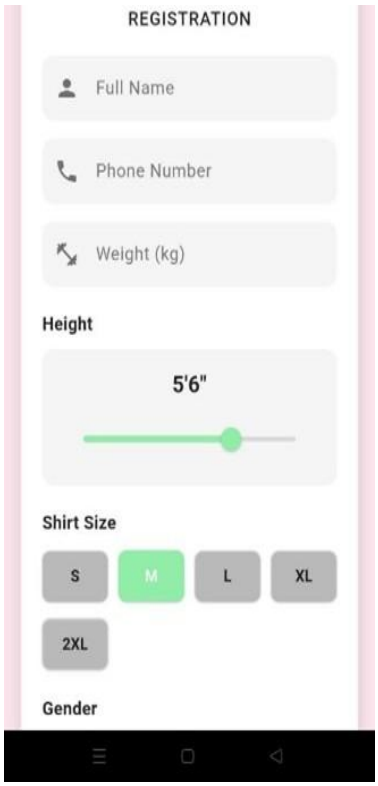

Figure 6: User Registration

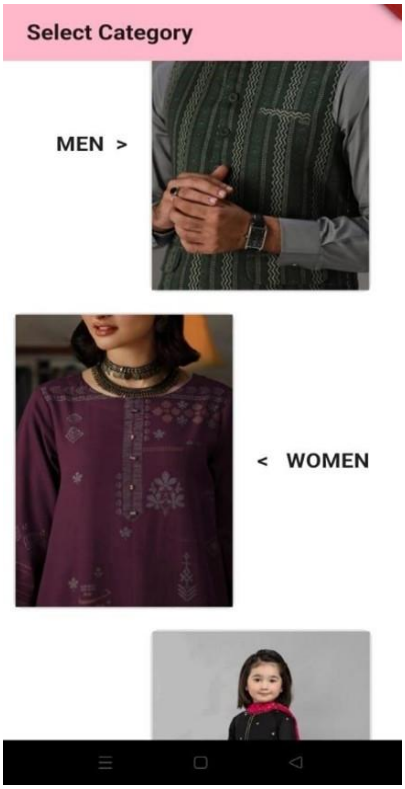

Figure 7: User select Category

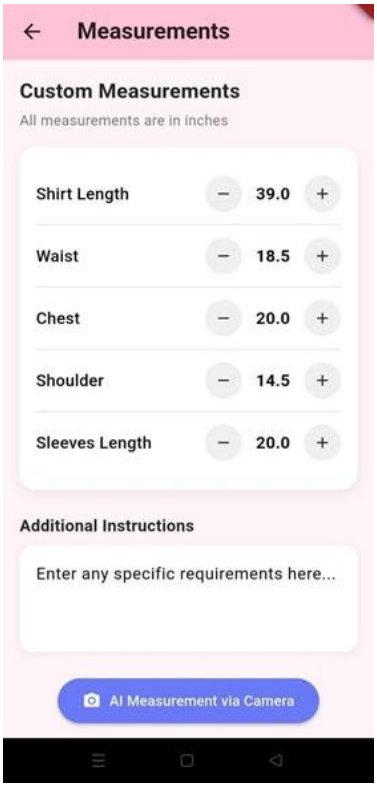

Figure 8: Manual Measurements

Figure 9: User clicking Image

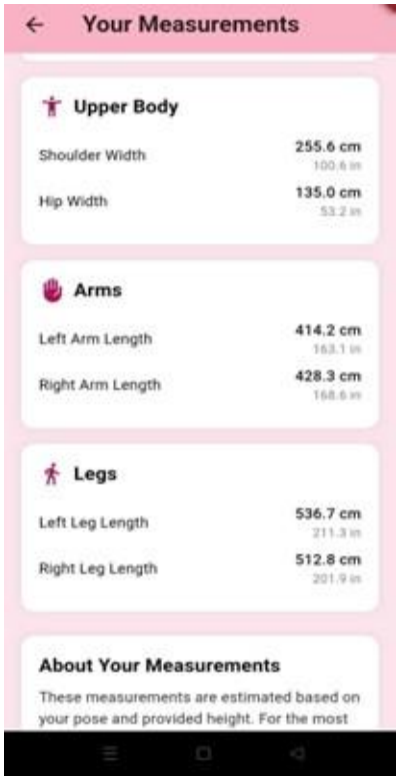

Figure 10: Displaying Result

Figure 11: Recomended Dresses

3.3.2. Screen Objects and Actions

Table 4: Measurement Capture Screen

Screen Object	Description	Action Performed
Capture Button	Button used to initiate image capture	Activates the device camera to capture body image
Retake Button	Allows user to discard current image	Clears captured image and reopens camera
Confirm Button	Confirms captured image	Sends image to pose detection and measurement module
Back Button	Navigation control	Returns user to home screen

Table 5: Recommendation & Results Screen

Screen Object	Description	Action Performed
Measurement Result Panel	Displays calculated body measurements	Shows height, width, and other computed values
Recommendation Button	Triggers recommendation process	Generates clothing size and design suggestions
History Button	Access point for previous records	Opens measurement history screen
Save Button	Stores current results	Saves measurements and recommendations in database

3.4 Behavioural Model

Interaction Diagrams:
The user initiates measurement through the UI, which triggers the CameraHandler to capture an image. The PoseDetectionService processes the image and forwards landmarks to the MeasurementEngine. The calculated measurements are stored in Firebase and passed to the RecommendationEngine to generate personalized clothing suggestions.

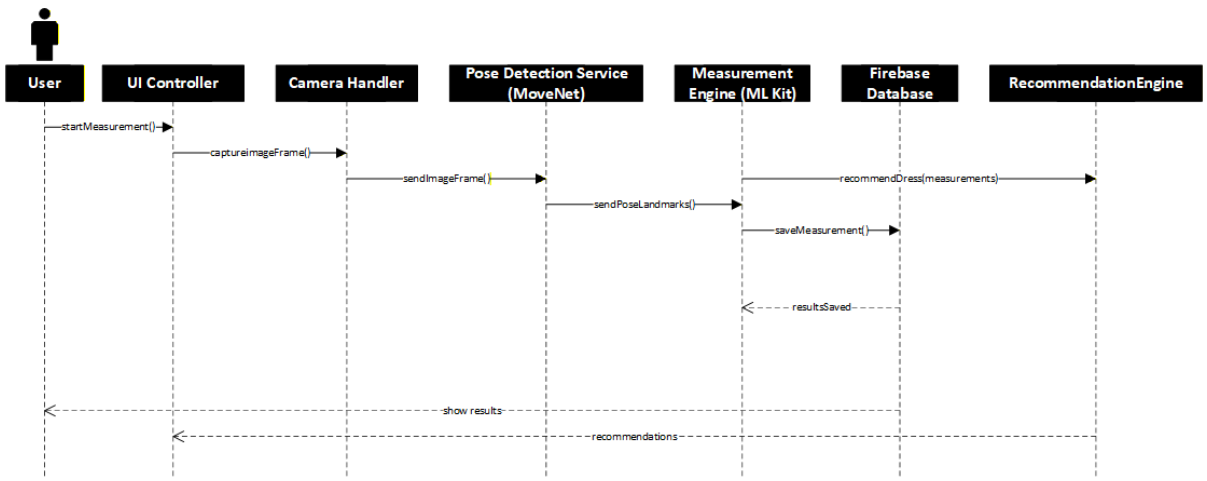

Figure 12: Sequence diagram Behavioural Model.

State Diagrams:
The system moves from App Idle to Login, activates the camera, captures the image, performs pose detection and measurement calculation, displays results, saves data, generates recommendations, and finally terminates the session.

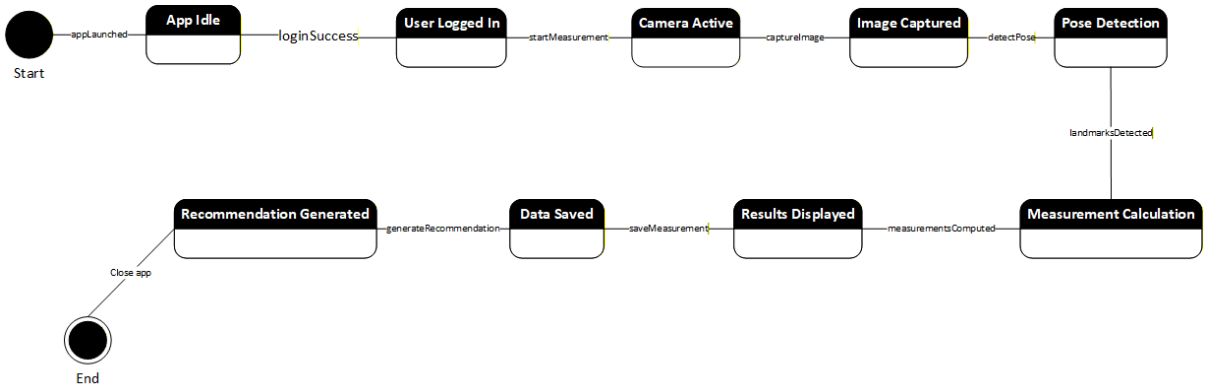

Figure 13: State transitions within the software.

4. Design Decisions

During the design of the AI-Based Body Measurement App, several critical decisions were made to ensure the system is **accurate, efficient, and maintainable**. Below is a summary of the key design choices, the approaches used, and the reasoning behind each choice.

Table 6: Summary of the key design choices

Design Choice	Approach Used	Reason	Benefit
Object-Oriented Design	Object-Oriented Design (OOD)	Models real-world entities (User, PoseLandmark, MeasurementRecord) for modularity and reuse.	Enhances maintainability, reusability, and clarity.
Architectural Pattern	Layered + Client-Server	Separates Presentation, Application Logic, AI Processing, and Data layers; supports client-server communication.	Improves modularity, security, and component interaction.
AI Integration	MoveNet for Pose Detection; ML Kit for Measurement Calculation	MoveNet provides accurate real-time pose estimation; ML Kit enables on-device calculations.	Ensures speed, accuracy, and preserves user privacy.
Database Design	NoSQL (Firebase Firestore / Realtime Database)	Flexible and scalable storage for users, measurements, and session history; normalized collections.	Efficient data storage/retrieval and offline access.
Measurement Calculation	Algorithmic computation using pose landmarks	Calculates height, chest, waist, hip directly from body joints.	Reduces manual errors; ensures reliable measurements.

User Interface Design	Flutter framework	Cross-platform support; easy integration with camera and Firebase.	Smooth, interactive, user-friendly app experience.

5. Summary

In this chapter, the key design ideas of the AI-Based Body Measurement App were discussed and refined to ensure the system meets its objectives efficiently and accurately. The **object-oriented design** was chosen to model real-world entities, while the **layered client-server architecture** ensures clear separation of concerns among the user interface, application logic, AI processing, and data storage.

Critical design decisions, including the use of **MoveNet for pose detection**, **ML Kit for measurement calculation**, and **Firebase for flexible database management**, were made to maximize accuracy, speed, and user privacy. The chapter also highlighted algorithmic approaches for body measurement calculation, UI design choices using Flutter, and component interactions to ensure smooth workflow and maintainability.

Overall, the design decisions and refinements presented in this chapter provide a **solid foundation for implementing the system**, ensuring that all project objectives—accurate body measurement, user-friendly interface, data management, and maintainability—are fully addressed.

References

Books:

Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*. 4th Edition. Boston, MA: Pearson, 2020, pp. 45-102.

Fowler, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd Edition. Boston, MA: Addison-Wesley, 2003, pp. 25-60.

Evans, B. *Flutter in Action: Build Native Mobile Apps with Flutter*. Shelter Island, NY: Manning Publications, 2021, pp. 10-50.

Articles / Journals

Pavllo, M., et al. "3D Human Pose Estimation in Video with Temporal Convolutions and Semi-Supervised Training." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 5, pp. 3600-3614, 2022.

Zhang, K., et al. "MoveNet: Real-Time Human Pose Estimation on Mobile Devices." *Proceedings of the CVPR Workshop on AI in Mobile Systems*, 2021, pp. 12-20.

Conference Proceedings

Kingma, D., & Ba, J. "Adam: A Method for Stochastic Optimization." *Proc. 3rd International Conference on Learning Representations (ICLR)*, 2015, pp. 1-15.

World Wide Web

Google Developers. "ML Kit Pose Detection." Internet: https://developers.google.com/ml-kit/vision/pose-detection, Updated: Oct 12, 2023 [Accessed: Dec 27, 2025].

Firebase. "Cloud Firestore Documentation." Internet: https://firebase.google.com/docs/firestore, Updated: Nov 15, 2023 [Accessed: Dec 27, 2025].

Flutter. "Flutter Official Documentation." Internet: https://flutter.dev/docs, Updated: Dec 1, 2025 [Accessed: Dec 27, 2025].